

Strategi för systemrobusthet: Arkitektur för integritet och motståndskraft i ACME

ACME-plattformens fundamentala syfte är att transformera probabilistisk och opålitlig modellinferens till deterministiska arkitektoniska sanningar. För kritiska domäner som Narrative, Research och Legal är systemrobusthet ingen valfri teknisk egenskap, utan en affärsförutsättning där marginalen för osäkerhet är obefintlig. Genom att kapsla in icke-deterministiska AI-utdata i ett "hårt skal" av traditionell systemdesign (ACME-0001) garanterar vi att varje exekvering är spårbar, atomär och verifierbar, oavsett externa avbrott eller modellens fluktuationer.

1. Transaktionsmodellen: Den atomära enheten (Unit of Work)

För att eliminera risken för partiella tillstånd och datainkonsistens tillämpar ACME en strikt transaktionsmodell genom mönstret "Unit of Work". Den strategiska kärnan i detta mönster är att säkerställa att systemets interna tillstånd och dess externa påverkan alltid förblir synkroniserade. Baserat på ADR 0003 och 0006 implementeras detta via ExecutionRepository, som utnyttjar SQLite i WAL-läge (Write-Ahead Logging). Genom kommandot BEGIN IMMEDIATE tvingas en strikt **Single-Writer Constraint** fram, vilket är den primära garantin för att undvika korruption vid samtidiga skrivningar. Arkitekturen kulminerar i en **atomär promotion**, där följande element kommittas som en odelbar enhet:

- **State:** Den reviderade tillståndsmaskinen (Revisioned Snapshot).
- **Memory:** Nya och uppdaterade minnesposter.
- **Documents:** Permanenta artefakter (t.ex. juridiska analyser eller prosa).
- **Events:** Domänhändelser för interna system.
- **Outbox-rader:** Den enda tillåtna mekanismen för att synkronisera sidoeffekter med internt tillstånd. "So what"-aspekten av denna atomicitet är kritisk: om ExecutionRepository misslyckas med att persistera en PreparedCommit, rullas samtliga förändringar tillbaka. Detta eliminerar helt risken för "Ghost Side-Effects", där externa händelser publiceras trots att det interna tillståndet gått förlorat. För att säkerställa identiska beteenden i test och produktion tillämpas en "copy-on-commit"-regel för in-memory-atomicitet, vilket speglar den hållbara SQLite-implementeringen. Denna transaktionella integritet vilar i sin tur på en orubblig identitetsarkitektur.

2. Idempotens och Identitetsarkitektur

I distribuerade system är deterministisk identitet förutsättningen för säker återupptagning. ACME tillämpar en identitetsarkitektur som garanterar att samma logiska operation alltid resulterar i samma identitet, utan att belasta globala ID-generatorer (ADR 0012). | Algoritm | Syfte || ----- | ----- || **acme-execution-id-1** | Härleder exekverings-ID baserat på initial kontext; ingen konsumtion av globala generatorer. || **acme-operation-key-1** | Identifierar en unik logisk operation inom en specifik exekvering. || **acme-transition-id-1** | Binder en tillståndsovergång till en operation (ADR 0004). |

För att hantera idempotens används **acme-request-fingerprint-1**, som genereras via SHA-256 över en kanonisk **acme-cjson-1**-representation av den validerade uppgiftsdatan (Task Input). Medan budgetar och policyer exkluderas från fingerprintet för att tillåta operativa justeringar, tvingar varje ändring i uppgiften fram en ny identitet. Systemets

integritet skyddas av operationDigest (baserat på **acme-operation-digest-1**). Om systemet vid en omstart försöker skriva divergent innehåll under en redan existerande identitet, utlöses ett terminalt **PERSISTENCE_CORRUPTION** . Detta är en "stop-the-world"-händelse som förhindrar att inkonsekventa data förgiftar exekveringsloggen. Dessa stabila identiteter möjliggör i sin tur ACME:s resume-mekanism.

3. Durable Execution Resume: Återhämtning efter krasch

Ekonomiska och operativa risker vid förlust av resultat från dyra modellanrop hanteras genom *Durable Execution Resume* . Syftet är att aldrig betala för samma inferens två gånger. Enligt ADR 0017 följer systemet en strikt beslutsmatris vid återupptagning: | Tillstånd | Åtgärd | Motivering || ----- | ----- | ----- || **Ingen reservation** | Kör om | Inget anrop har skickats; säkert att starta om. || **Succeeded** | Fortsätt | Svar finns lagrat; använd befintlig evidens. || **Reserved / In-flight** | **MODEL_UNAVAILABLE** | Risk för dubbeldebitering/okänd status; terminalt fel. || **Unrecoverable** | **RESUME_EVIDENCE_UNAVAILABLE** | Evidens (t.ex. kryptering) saknas; kan ej återställas. |

En central teknisk garanti är **Context Re-read** . Vid resume läser systemet om State och Memory (loadContext) istället för att förlita sig på det gamla read-setet. Detta tvingar fram **Optimistic Concurrency Control** : om en annan skribent har flyttat entitetens revision under tiden systemet var nere, nekas commit med felkoden CONFLICT_STATE_REVISION. Varje resume-försök loggas med ett nytt sekventiellt försöksnummer (ADR 0017), vilket säkerställer att audit-loggen reflekterar avbrottet snarare än att dölja det.

4. Outbox-mönstret: Garanterad leverans av sidoeffekter

För att lösa utmaningen med "dual writes" tillämpas Outbox-mönstret, vilket garanterar *at-least-once* leverans av sidoeffekter (t.ex. bildgenerering). Livscykeln styrs av stegen **Lease** , **Deliver** och **Settle** (ADR 0018). Termen "Lease" används istället för "Claim" för att respektera ACME:s **Core Vocabulary Guard** , vilket förhindrar domänläckage från Research-domänen in i motorn. Tekniskt används **Visibility Timeouts** (lease-expiration) som återhämtningsmekanism vid krascher. Om en arbetare dör under leverans, löper timeouten ut och meddelandet blir återigen tillgängligt, vilket skapar ett självläkande system utan behov av komplexa distribuerade lås. Viktigt är att ExecutionRepository är en passiv lagringsenhet utan egen intelligens kring retries. Retry-policyn injiceras som en funktion från **Composition Root** , vilket ger operatören full kontroll över backoff-strategier. Statusen failed är terminal och kräver manuell intervention, vilket säkerställer att kritiska leveransfel inte ignoreras.

5. Verifiering och Replay: Systemets integritetsgaranti

Replay-kapabiliteten är den ultimata garantin för systemets korrekthet och audit-stabilitet. Genom replayVerify rekonstrueras en exekvering genom att jämföra den lagrade **acme-operation-digest-1** med en version som beräknas på nytt baserat på sparad evidens. Till skillnad från en vanlig körning använder replay det **inspelade read-setet** snarare än aktuella databasvärden. Detta bevisar vad systemet beslutade vid den specifika tidpunkten, även om världen har förändrats sedan dess. Utfallet klassificeras enligt följande:

- **Match**: Den återskapade digesten är identisk. Systemets logik är intakt.
- **Different**: En avvikelse har uppstått i logik eller determinism. Indikerar ett behov av teknisk analys.

- **Unavailable:** Nödvändig evidens saknas, t.ex. vid **hash-only** retention där rådata raderats. Fullständig verifiering (match) kräver retentionspolicyn **encrypted-payload** (ADR 0016), vilket är den enda nivån som bevarar rådata för live-anrop i ett format som kan dekrypteras för framtida replay.

6. Slutsats och framtida motståndskraft

ACME:s arkitektur för systemrobusthet skapar en obruten kedja av evidens och atomicitet. Genom att kombinera transaktionell integritet (ADR 0003), deterministisk identitet (ADR 0012), robust återhämtning via Resume (ADR 0017) och garanterad Outbox-leverans, har vi byggt en plattform som inte bara hanterar osäkerhet, utan tämjer den. Systemet är självdokumenterande och motståndskraftigt mot både systemfel och oförutsägbara modellbeteenden, vilket gör det redo för de mest krävande uppdragen i komplexa domäner.